Prácticas de índices — MongoDB 8.0

Laboratorio 1 — Una misma venta, dos formas de encontrarla

Objetivo: encontrar una venta concreta entre 200.000 documentos, medir la búsqueda sin índice y repetirla después de crearlo. Si nunca has utilizado Docker ni `mongosh`, puedes seguir los pasos sin adivinar qué consola utilizar.

Antes de empezar: las tres piezas del laboratorio

- **Terminal:** PowerShell en Windows o Terminal en macOS/Linux. Aquí se escriben las órdenes `docker compose`.
- **Contenedor:** entorno aislado que ejecuta MongoDB. En este curso el servicio se llama `mongo-indices`.
- **mongosh:** consola que interpreta instrucciones de MongoDB. La ejecutaremos *dentro* del contenedor; no tienes que instalarla aparte.

Trabajaremos solo con documentos sintéticos; el archivo de carga **borra y recrea** la colección `curso_indices.ventas`. No uses estos scripts sobre bases de datos personales o empresariales.

Paso 1 — Descargar el proyecto

Abre https://github.com/jaimecopilot/MONGODB, pulsa **Code → Download ZIP** y descomprímelo. Abre PowerShell o Terminal en su carpeta principal. Debes ver `compose.yaml` y `.env.example`. Si no los ves, probablemente has abierto una carpeta equivocada. En PowerShell puedes comprobarlo con `Get-ChildItem`; en Bash, con `ls`.

Paso 2 — Preparar las variables locales

En PowerShell:

```
Copy-Item .env.example .env
```

En macOS/Linux:

```
cp .env.example .env
```

Abre `.env` y cambia `MONGO_ADMIN_PASSWORD` por una contraseña **solo para este laboratorio**. Aunque el servidor de índices no requiera autenticación, Docker Compose procesa la configuración de los otros servicios. No publiques `.env`.

Paso 3 — Arrancar MongoDB

En la **terminal de tu ordenador**, no en MongoDB:

```
docker compose up -d --wait mongo-indices
docker compose ps
```

`up` crea o inicia el contenedor; `-d` lo deja funcionando en segundo plano; `--wait` comprueba que esté listo. La lista de `docker compose ps` debería mostrar `mongo-indices` en ejecución.

Si Docker no responde: abre Docker Desktop y comprueba que esté iniciado. Si recibes un mensaje de puerto ocupado, puede existir otro MongoDB local en el puerto 27017. No detengas servicios desconocidos: consulta al formador.

Paso 4 — Crear 200.000 ventas

```
docker compose exec -T -e VENTAS=200000 mongo-indices mongosh --quiet --file /lab/
01_cargar_ventas.js
```

La instrucción `exec` entra en un contenedor que ya existe. `VENTAS=200000` indica la cantidad de registros. `--file` indica a `mongosh` qué script ejecutar. La carpeta `/lab` está **dentro del contenedor** y corresponde a la carpeta `lab` del proyecto.

Espera a que aparezca la confirmación final. Puedes comprobar el número de registros con:

```
docker compose exec -T mongo-indices mongosh --quiet --eval
'db.getSiblingDB("curso_indices").ventas.countDocuments({})'
```

Resultado esperado: `200000`. Si obtienes cero, vuelve a leer los mensajes de la carga antes de continuar.

Paso 5 — Ver que la venta existe

```
docker compose exec -T mongo-indices mongosh --quiet --eval
'printjson(db.getSiblingDB("curso_indices").ventas.findOne({numero:199998}))'
```

Debe aparecer un documento con `numero: 199998`. ¿Por qué hemos hecho esto? Para separar dos preguntas: **¿es correcto el resultado?** y **¿cuánto cuesta obtenerlo?**. La primera ya tiene respuesta. Ahora investigaremos la segunda.

Paso 6 — Medir ANTES del índice

```
docker compose exec -T mongo-indices mongosh --quiet --file /lab/02_antes.js
```

Este script se niega a trabajar si ya existen índices secundarios, para que no confundas un resultado preparado de antemano con la línea base.

Busca en su salida `COLLSCAN` y los contadores `nReturned`, `totalDocsExamined` y `totalKeysExamined`. Escribe los valores:

- Documentos encontrados: _____
- Documentos examinados: _____
- Claves examinadas: _____
- Etapa de búsqueda: _____

Interpretación esperada: encontrar una venta puede requerir examinar una parte enorme de la colección. No supongas que `nReturned=1` significa «se leyó un documento».

Paso 7 — Crear el índice

```
docker compose exec -T mongo-indices mongosh --quiet --file /lab/03_crear_indices.js
```

Este script crea `ix_numero`, un índice único ascendente sobre `numero`, además de otros índices que servirán para clases posteriores. La instrucción esencial que aprenderemos es:

```
db.ventas.createIndex({numero:1},{name:"ix_numero",unique:true})
```

`unique` exige que los números no se repitan. Esa restricción protege la integridad y es distinta del beneficio de búsqueda.

Paso 8 — Medir DESPUÉS del índice

```
docker compose exec -T mongo-indices mongosh --quiet --file /lab/04_despues.js
```

Compara la salida con el paso 6. El script verificará que ahora la búsqueda por `numero` examina un documento. En el plan debes poder identificar `IXSCAN`.

Dato	Antes	Después
Documentos encontrados	___	___
Documentos examinados	___	___
Claves examinadas	___	___
Plan predominante	___	___

Pregunta esencial: si devolvemos la misma venta, ¿qué hemos ganado? Respuesta: podemos haber reducido enormemente el esfuerzo de localización. Los milisegundos son orientativos y dependen del ordenador.

Paso 9 — Si algo no coincide

- `02_antes.js` informa de índices previos: ejecutaste el laboratorio antes. Repite el paso 4 para recuperar una colección sin índices; recuerda que la carga borra datos docentes e índices secundarios.
- No aparece el documento: comprueba que cargaste 200.000 ventas y buscaste por `numero`, sin tilde.
- `04_despues.js` falla: comprueba que completaste el paso 7 y que utilizas el mismo servicio `mongo-indices`.
- Error de conexión: ejecuta `docker compose ps` y después `docker compose logs --tail=80 mongo-indices`.

Paso 10 — Conclusión y cierre

Escribe tres frases: (1) qué consulta hiciste; (2) qué trabajo realizó antes del índice; (3) qué cambió después. No basta con afirmar «el índice es más rápido».

Cuando termines:

```
docker compose stop mongo-indices
```

Este comando detiene el contenedor. No uses `docker compose down --volumes` salvo que quieras destruir los datos y volúmenes del laboratorio.

Autoevaluación: ¿qué ocurriría si ejecutaras la carga del paso 4 después de haber creado el índice? ¿Por qué es importante respetar el orden de esta práctica?

Laboratorio 2 — Índices compuestos, orden de campos y ESR

Qué vamos a hacer

La persona responsable de ventas quiere **las diez ventas pagadas más recientes que superan los 500 euros**. Nuestra tarea no es copiar índices al azar. Vamos a escribir la consulta, comprobar que los resultados cumplen el encargo, elegir dos índices candidatos y comparar el trabajo real que exigen.

Duración: esta actividad no está limitada a una cifra fija de minutos. Avanza cuando hayas comprendido el resultado de cada paso.

Requisitos y advertencias

Debes tener el repositorio del curso en tu ordenador, Docker en funcionamiento y la instancia `mongo-indices` preparada.

Esta práctica **continúa la anterior**: allí cargaste 200.000 ventas y ejecutaste `03_crear_indices.js`. Si acabas de empezar en un entorno limpio, abre la práctica 1 y sigue sus pasos de preparación, carga y creación de índices **antes** de continuar.

Nunca vuelvas a ejecutar `01_cargar_ventas.js` a mitad de esta práctica: borrarías la colección y perderías sus índices, cambiando el experimento.

PASO 1 — Comprobar que estamos trabajando en el lugar correcto

Abre PowerShell, Terminal o Bash **en la carpeta del repositorio** y ejecuta:

```
docker compose ps
```

Debes ver el servicio `mongo-indices` en ejecución. Si está detenido:

```
docker compose up -d --wait mongo-indices
```

No necesitas entrar a mano en `mongosh` para estos primeros pasos: pasaremos expresiones mediante `--eval` desde la terminal.

Comprueba el número de ventas:

```
docker compose exec -T mongo-indices mongosh --quiet --eval
'db.getSiblingDB("curso_indices").ventas.countDocuments({})'
```

Resultado esperado: `200000` si mantuviste la carga de la primera práctica. Si aparece cero, aún no hay datos. Si aparece otro número que tú preparaste, puedes continuar, pero no compares tus cifras absolutas con las de un compañero con 200.000 ventas.

PASO 2 — Confirmar que los campos existen

Pide a MongoDB un documento de ejemplo:

```
docker compose exec -T mongo-indices mongosh --quiet --eval
'printjson(db.getSiblingDB("curso_indices").ventas.findOne({estado:"pagada"}))'
```

Localiza `estado`, `fecha` e `importe`. Observa que:

- `estado` es una cadena con valores como `pagada`.
- `fecha` es una fecha almacenada como tipo fecha.
- `importe` es numérico, de modo que podemos hacer comparaciones mayores/menores.

No avances si alguno de esos campos no aparece: puede que estés en otra base de datos o que uses un conjunto de documentos diferente.

PASO 3 — Traducir el pedido de negocio a una consulta

El encargo tiene cuatro partes:

Requisito	Expresión MongoDB
Solo pagadas	`estado:"pagada"`
Importe superior a 500	`importe:{$gt:500}`
Más recientes primero	`sort({fecha:-1})`
Solo diez	`limit(10)`

Para no confundir la consulta con el comando de Docker, la ejecutaremos como archivo en el paso siguiente. Antes, intenta leer mentalmente esta expresión:

```
db.ventas
  .find({estado:"pagada",importe:{$gt:500}})
  .sort({fecha:-1})
  .limit(10)
```

Si dices «busca todas las ventas y luego imprime las que quiera», todavía falta precisión: la consulta filtra por **dos condiciones** y ordena antes de limitar.

PASO 4 — Contar cuántas ventas cumplen las condiciones

Ejecuta en tu terminal:

```
docker compose exec -T mongo-indices mongosh --quiet --eval
'print(db.getSiblingDB("curso_indices").ventas.countDocuments({estado:"pagada",importe:{$gt:
500}}))'
```

La cifra representa todas las ventas pagadas que superan el umbral. **No tienen por qué ser solo diez**: el límite de diez se aplica después, cuando pidamos una lista de las más recientes.

Escribe el número obtenido: _____.

¿Por qué hacemos esta comprobación? Si apenas hubiera tres documentos que cumplen la condición, elegir un índice para ordenar millones de candidatos sería una preocupación muy distinta a la de una colección con miles de coincidencias.

PASO 5 — Reconocer el índice que ya existe

El archivo `lab/03_crear_indices.js` crea, entre otros, este índice:

```
{estado:1,fecha:-1,importe:1}
```

Su nombre en nuestro proyecto es `ix_esr`. Comprueba su existencia:

```
docker compose exec -T mongo-indices mongosh --quiet --eval
'printjson(db.getSiblingDB("curso_indices").ventas.getIndexes().filter(x=>x.name==="ix_esr"))'
```

Deberías ver una definición que contiene `estado`, `fecha` e `importe`, en ese orden. Si la salida es `[]`, el índice no existe. Vuelve a los pasos de la práctica anterior: probablemente no ejecutaste `03_crear_indices.js` o recargaste después las ventas.

PASO 6 — Comparar orden ESR y ERS

Vamos a utilizar un script nuevo:

```
docker compose exec -T mongo-indices mongosh --quiet --file /lab/15_compuestos_guiado.js
```

Antes de ejecutarlo, comprende qué hará:

1. Verificará que existe la colección y el índice `ix_esr`.
2. Creará el índice alternativo `ix_ers_practica` con `{estado:1,importe:1,fecha:-1}`.
3. Ejecutará **la misma consulta** dos veces, una obligando a usar cada índice mediante `hint`.
4. Comprobará que ambas variantes devuelven diez ventas que cumplen los filtros y que las fechas están ordenadas de manera descendente. Si varias ventas comparten fecha, **los identificadores concretos pueden variar legítimamente**: la consulta no ha establecido cómo resolver los empates.
5. Imprimirá un resumen de `executionStats` para poder comparar el trabajo efectuado.

El script no borra las ventas. Añade un índice de laboratorio, que puedes consultar o eliminar más tarde.

Cómo leer el resultado

Busca dos bloques etiquetados `ESR` y `ERS`. En cada uno anota:

Dato	ESR	ERS
`nReturned`	___	___

Dato	ESR	ERS
`totalDocsExamined`	____	____
`totalKeysExamined`	____	____
¿Aparece una etapa `SORT`?	____	____
`executionTimeMillis`	____	____

Importante: no te he dado números fijos de tiempo; no sería riguroso prometerlos. Los valores obtenidos son los de tu ejecución, no una puntuación a aprobar.

PASO 7 — Entender la diferencia sin memorizar la sigla

El índice ESR agrupa primero por `estado`, dentro de ese grupo organiza `fecha` en orden descendente y deja `importe` como tercera clave. Eso facilita recuperar documentos ya ordenados por fecha cuando conoces el estado, aunque el rango de importes no se pueda reducir tan eficazmente como cuando aparece antes.

El índice ERS agrupa por `estado` y después por `importe`. Puede localizar un rango de importes más directamente, pero el orden global por `fecha` entre distintos importes puede requerir una ordenación adicional.

Pregunta clave: ¿cuál ha examinado menos claves? ¿Cuál evita una operación de ordenación adicional? ¿Es el mismo índice el mejor en ambas métricas? Elige según los datos y el uso, no según un eslogan.

PASO 8 — Comprobar que ningún índice ha cambiado la respuesta

El script confirma que **ambas variantes obedecen la consulta**: diez ventas pagadas, con importe superior a 500, en orden descendente de fecha.

¿Por qué importa esa comprobación? Porque una optimización **no puede cambiar el significado de la consulta**. Debe seguir representando «diez ventas pagadas de más de 500 euros, ordenadas de más reciente a más antigua». Eso no garantiza que el límite seleccione los mismos identificadores entre ventas empatadas en fecha.

Sutileza importante: varias ventas pueden compartir exactamente la misma fecha. `sort({fecha:-1})` no especifica qué venta debe aparecer primero dentro de un empate. Si los dos planes eligen documentos diferentes con la misma fecha límite, eso no demuestra pérdida de datos ni un error de índice. Para un orden total estable podrías pedir `sort({fecha:-1,numero:1})`, pero entonces también tendrías que estudiar índices que incluyan ese segundo campo de ordenación. Lo haremos en una práctica posterior.

PASO 9 — Errores habituales

- **El script dice que falta `ix_esr`:** ejecuta la práctica anterior hasta el paso de creación de índices.

- **Dice que no hay ventas:** completa la carga de datos; no crees índices sobre una colección vacía fingiendo un experimento de 200.000 registros.
- **Aparece otra cantidad de resultados:** comprueba el estado y el filtro por importe, y que consultaste el servidor correcto.
- **Hay una operación `SORT` en ERS:** no es un fallo del script; es precisamente una característica posible del plan.
- **El tiempo cambia al repetirlo:** es normal; por eso usamos también los contadores de documentos y claves.
- **Falló la creación del índice:** revisa el texto del error y el espacio disponible. No borres todos los índices ni la colección para solucionar un error que no comprendes.

PASO 10 — Retomar el control

Al terminar, puedes examinar los dos índices:

```
docker compose exec -T mongo-indices mongosh --quiet --eval
'printjson(db.getSiblingDB("curso_indices").ventas.getIndexes().filter(x=>["ix_esr","ix_ers_pra
ctica"].includes(x.name)))'
```

No borres `ix_esr` porque forma parte de los ejercicios siguientes. Si deseas eliminar exclusivamente el índice creado para la comparación:

```
docker compose exec -T mongo-indices mongosh --quiet --eval
'db.getSiblingDB("curso_indices").ventas.dropIndex("ix_ers_practica")'
```

Después puedes detener `mongo-indices` con `docker compose stop mongo-indices`. Si vas a continuar, déjalo funcionando.

Actividad de comprensión

Explica, con un ejemplo de tus ventas, cómo responderías a alguien que afirma:

> «Siempre hay que poner primero el campo más selectivo y dejar la fecha al final».

Tu respuesta debe distinguir igualdad, rango, ordenación y el volumen de candidatos. No se acepta como justificación repetir simplemente «porque lo dice ESR».

Siguiente tema: un índice multikey para un campo que contiene una lista de etiquetas, y por qué ese índice es distinto de uno sobre un valor escalar.

Laboratorio 3 — Buscar etiquetas de ventas y palabras dentro de descripciones

Qué aprenderás y por qué importa

La primera práctica localizaba una venta mediante un número. Pero una tienda también puede pedir «todas las ventas etiquetadas como urgentes» o «descripciones que hablen de café». Son problemas distintos de la búsqueda exacta por número.

El objetivo es comprender cómo MongoDB busca valores dentro de arrays, qué significa un índice multikey y por qué un índice de texto clásico no equivale a buscar un campo por igualdad.

Paso 1. Verificar datos y conexión

Abre PowerShell o Bash dentro de la carpeta del repositorio. Si detuviste MongoDB:

```
docker compose up -d --wait mongo-indices
```

Comprueba que la colección `curso_indices.ventas` contiene datos:

```
docker compose exec -T mongo-indices mongosh --quiet --eval
'print(db.getSiblingDB("curso_indices").ventas.countDocuments({}))'
```

Si devuelve cero, vuelve a la primera práctica y carga las ventas ficticias. No continúes interpretando una búsqueda sobre datos inexistentes.

Paso 2. Observar qué significa tener un array

```
docker compose exec -T mongo-indices mongosh --quiet --eval
'printjson(db.getSiblingDB("curso_indices").ventas.findOne({numero:1}))'
```

Localiza la propiedad `etiquetas`. En el conjunto sintético tiene varios valores, por ejemplo una etiqueta general y otra que indica si la venta procede de la web. Algunos documentos incluyen `urgente`.

Pregunta previa: ¿crees que `etiquetas` es una cadena única como `"web,urgente"` o una lista de valores separados? La diferencia es importante: MongoDB reconoce sus elementos como valores independientes dentro de un array.

Paso 3. Consultar un valor contenido en la lista

```
docker compose exec -T mongo-indices mongosh --quiet --eval
'print(db.getSiblingDB("curso_indices").ventas.countDocuments({etiquetas:"urgente"}))'
```

La consulta no pide que el array sea exactamente igual a `"urgente"`. Pide documentos donde ese valor aparezca como elemento del campo array. Anota el número de coincidencias: _____.

Paso 4. Verificar que existe el índice multikey

En la primera práctica, `03_crear_indices.js` creó `ix_etiquetas`:

```
docker compose exec -T mongo-indices mongosh --quiet --eval
'printjson(db.getSiblingDB("curso_indices").ventas.getIndexes().filter(i=>i.name==="ix_etiqueta
s"))'
```

Deberías observar la clave `etiquetas:1`. Como el campo contiene arrays en nuestros documentos, el índice funciona como multikey.

Ahora ejecuta una consulta forzando el índice **solo para estudiar el plan**:

```
docker compose exec -T mongo-indices mongosh --quiet --eval
'printjson(db.getSiblingDB("curso_indices").ventas.find({etiquetas:"urgente"}).hint("ix_etiquet
as").limit(3).explain("executionStats").executionStats)'
```

Busca `nReturned`, `totalDocsExamined` y `totalKeysExamined`. Pueden existir más claves que resultados: los arrays contribuyen a las entradas del índice. No confundas eso con documentos duplicados almacenados.

Paso 5. Comprender por qué dos condiciones pueden pertenecer a elementos diferentes

Imagina la siguiente venta:

```
{
  productos: [
    { sku: "SKU10", cantidad: 1 },
    { sku: "SKU20", cantidad: 3 }
  ]
}
```

Si buscamos por `productos.sku:"SKU10"` y `productos.cantidad:3` como dos condiciones independientes, cada condición puede satisfacerse en elementos distintos de ese array.

Para expresar que **el mismo elemento** debe tener ambas propiedades, usamos:

```
{
  productos: {
    $elemMatch: { sku: "SKU10", cantidad: 3 }
  }
}
```

La segunda consulta no debería encontrar el documento mostrado, porque el producto SKU10 tiene cantidad 1 y no 3. Este ejemplo demuestra una diferencia de significado, no simplemente un truco para optimizar rendimiento.

Paso 6. Probar la distinción sin alterar las ventas

Este experimento utiliza una colección temporal diferente de `ventas`. Entra en `mongosh`:

```
docker compose exec mongo-indices mongosh --quiet
```

Dentro de `mongosh` introduce estas instrucciones de una en una:

```
const d = db.getSiblingDB("curso_indices");
d.productos_ejemplo.drop();
d.productos_ejemplo.insertOne({
  _id: 1,
  productos: [
    {sku:"SKU10",cantidad:1},
    {sku:"SKU20",cantidad:3}
  ]
});
```

La llamada `drop()` elimina **solo la colección de prueba `productos_ejemplo`**. No afecta a `ventas`. Ahora comprueba:

```
d.productos_ejemplo.find({
  "productos.sku":"SKU10",
  "productos.cantidad":3
}).toArray()
```

Observa que la primera consulta puede encontrar el documento: la existencia de SKU10 y la existencia de cantidad 3 se verifican sin obligar a que correspondan al mismo elemento.

Después:

```
d.productos_ejemplo.find({
  productos:{$elemMatch:{sku:"SKU10",cantidad:3}}
}).toArray()
```

El resultado debe ser un array vacío. **Si entiendes por qué las salidas difieren, ya has comprendido la razón de utilizar `$elemMatch`.**

Puedes salir de `mongosh` escribiendo `exit`.

Paso 7. Pasar de etiquetas a palabras de una descripción

Ahora queremos buscar ventas cuya descripción incluya el término `cafe`. No lo buscamos como valor exacto del campo completo.

Comprueba primero que hay descripciones de café:

```
docker compose exec -T mongo-indices mongosh --quiet --eval
'printjson(db.getSiblingDB("curso_indices").ventas.findOne({descripcion:"cafe premium en
grano"}))'
```

La primera práctica crea también un índice de tipo `text` sobre `descripcion`. Comprueba su existencia mediante `getIndexes()` antes de utilizar el operador `$text`.

```
docker compose exec -T mongo-indices mongosh --quiet --eval
'print(db.getSiblingDB("curso_indices").ventas.countDocuments({$text:{$search:"cafe"}}))'
```

Resultado esperado: una cifra positiva. MongoDB busca el término mediante el mecanismo de búsqueda textual clásico. Esto no es lo mismo que `find({descripcion:"cafe"})`, que busca igualdad del valor completo.

Paso 8. Interpretar una puntuación textual

Si quieres observar la puntuación de coincidencia:

```
docker compose exec -T mongo-indices mongosh --quiet --eval
'printjson(db.getSiblingDB("curso_indices").ventas.find({$text:{$search:"cafe"}},{descripcion:
1,score:{$meta:"textScore"}}).sort({score:{$meta:"textScore"}}).limit(3).toArray())'
```

Busca `descripcion` y `score`. La puntuación sirve para ordenar relevancia textual según el mecanismo disponible; **no es una nota de calidad humana ni un porcentaje de certeza**.

Paso 9. Resolver errores sin improvisar

- **El operador `$text` dice que falta índice:** todavía no has ejecutado `03_crear_indices.js`, o recargaste ventas posteriormente.
- **`getIndexes()` devuelve solo `_id_`:** estás en la línea base de la primera práctica. Crea primero los índices.
- **La búsqueda de café devuelve cero:** examina una descripción real y revisa la configuración de idioma del índice; no supongas que cualquier búsqueda literal funciona igual.
- **`$elemMatch` no devuelve el documento:** en este ejemplo es el resultado correcto, no un error de MongoDB.

Tu explicación final

Redacta por qué estas tres consultas no son equivalentes:

```
{ etiquetas: "urgente" }
{ descripcion: "cafe" }
{ $text: { $search: "cafe" } }
```

Y explica qué diferencia de significado introduce `$elemMatch` cuando el campo contiene documentos anidados en un array.

Laboratorio 4 — Distancias, coordenadas e índice hashed

Propósito

La empresa tiene puntos de entrega y quiere responder «¿cuáles están cerca de esta dirección?». También necesita recuperar rápidamente ventas de un grupo concreto mediante una consulta por igualdad.

A primera vista ambos problemas parecen búsquedas, pero requieren estructuras distintas: `2dsphere` para determinadas consultas geográficas y `hashed` para igualdad sobre un valor transformado.

Paso 1. Preparar los datos

En la terminal de tu ordenador:

```
docker compose up -d --wait mongo-indices
```

Abre `mongosh` dentro del contenedor:

```
docker compose exec mongo-indices mongosh --quiet
```

A partir de este punto, los fragmentos JavaScript se escriben dentro de `mongosh`, no en PowerShell.

Paso 2. Crear tres ubicaciones conocidas

```
const d = db.getSiblingDB("curso_indices");
d.puntos_entrega_clase.drop();
d.puntos_entrega_clase.insertMany([
  {
    nombre:"Sol",
    ubicacion:{type:"Point",coordinates:[-3.7038,40.4168]}
  },
  {
    nombre:"Retiro",
    ubicacion:{type:"Point",coordinates:[-3.6833,40.4153]}
  },
  {
    nombre:"Barcelona",
    ubicacion:{type:"Point",coordinates:[2.1700,41.3800]}
  }
]);
```

`drop()` borra exclusivamente la colección de ejemplo `puntos_entrega_clase`. Las coordenadas están escritas como **longitud primero, latitud después**. Son puntos aproximados utilizados para explicar la consulta; no constituyen una aplicación de navegación real.

Paso 3. Crear el índice apropiado

```
d.puntos_entrega_clase.createIndex(
  {ubicacion:"2dsphere"},
  {name:"ix_puntos_esfera"}
);
```

El término `2dsphere` declara el tipo geoespacial adecuado para la geometría usada. Es diferente de un índice normal `{ubicacion:1}`, que no comprende por sí solo las operaciones sobre puntos terrestres.

Paso 4. Buscar puntos próximos

```
d.puntos_entrega_clase.find({
  ubicacion:{
    $near:{
      $geometry:{
        type:"Point",
        coordinates:[-3.7038,40.4168]
      },
      $maxDistance:3500
    }
  }
}).toArray()
```

Léela de fuera hacia dentro: buscar documentos, filtrar por ubicación cercana a un punto de referencia y exigir una distancia máxima aproximada de 3.500 metros.

Resultado esperado: Sol y Retiro; no Barcelona. Si la consulta encuentra también Barcelona, examina si intercambiaste las coordenadas o utilizaste una distancia absurda.

Paso 5. Demostrar qué problema NO resuelve el índice

Esta consulta no puede decirte automáticamente qué ruta debe seguir un repartidor, cuánto tráfico existe o cuánto tardará en llegar. El índice ayuda a buscar puntos próximos **geométricamente**. Calcular un trayecto por calles es otra aplicación y necesita otra información.

Anota: ¿qué diferencia hay entre «a 2 km en línea recta» y «a 2 km por carretera»?

Paso 6. Conocer los índices hashed

La colección de ventas incluye `idGrupo`. El archivo `03_crear_indices.js` añade un índice hashed sobre ese campo. Comprueba que existe:

```
d.ventas.getIndexes().filter(x=>x.name==="ix_grupo_hash")
```

Si aparece la definición con `"hashed"`, realiza una búsqueda por igualdad:

```
d.ventas.find({idGrupo:809}).hint("ix_grupo_hash").limit(3).toArray()
```

Debes ver ventas cuyo `idGrupo` sea 809. Ahora explica por qué este índice **no está pensado para recorrer de forma ordenada los grupos 800 a 900**: el hash representa valores sin conservar necesariamente su proximidad numérica original.

Paso 7. Interpretar el plan

```
d.ventas.find({idGrupo:809})
  .hint("ix_grupo_hash")
  .limit(3)
  .explain("executionStats")
```

Observa si el plan usa un índice y cuántas claves/documentos examina. `hint` se utiliza en esta demostración para estudiar un índice concreto. No es una recomendación de forzar siempre un acceso en producción.

Paso 8. Cerrar y justificar

Sal de `mongosh` escribiendo `exit`. Resume:

- qué pregunta resolvió `2dsphere`;
- qué significado tienen las coordenadas;
- qué pregunta resuelve bien `hashed`;
- por qué ninguna de las dos estructuras sustituye automáticamente a un índice ordenado convencional por número.

Error habitual: confundir «hashed» con contraseña cifrada. Un índice hashed organiza claves mediante un cálculo hash; no protege por sí mismo secretos personales.

Laboratorio 5 — Impedir duplicados y caducar sesiones: índices unique y TTL

Dos problemas que no son simplemente «buscar rápido»

La empresa tiene dos necesidades nuevas. Quiere impedir que un mismo correo electrónico aparezca repetido entre las cuentas de un ejercicio, y quiere que ciertos registros de sesión desaparezcan cuando llegue su fecha de caducidad. Son reglas sobre **integridad** y **ciclo de vida de datos**.

No debemos deducir que todo índice se crea solo para acelerar una consulta. Algunos índices también ayudan a aplicar reglas sobre qué datos pueden existir o cuándo deben eliminarse.

Paso 1. Preparación y consecuencias

Abre PowerShell o Bash en la raíz del repositorio. Si la instancia está detenida, arráncala:

```
docker compose up -d --wait mongo-indices
```

Este ejercicio utiliza dos colecciones separadas de las ventas principales:

- `curso_indices.unicidad_demo` para la prueba de duplicados;
- `curso_indices.sesiones_demo` para probar caducidad.

El script que vas a ejecutar borra y recrea esas dos **colecciones didácticas**. No borra `curso_indices.ventas`. Asegúrate de no haber guardado en ellas datos propios que quieras conservar.

Paso 2. Entender la regla de unicidad

El correo ficticio `unico@ejemplo.test` debe aparecer una sola vez. Dentro de `mongosh` podríamos crear el índice así:

```
db.getSiblingDB("curso_indices").unicidad_demo.createIndex(
  {email:1},
  {unique:true}
)
```

No significa «el campo email es obligatorio para cualquier aplicación». Significa que MongoDB aplicará la semántica de unicidad de ese índice a los documentos de esa colección. Si el campo puede faltar o ser nulo, el diseño requiere reglas adicionales que estudiaremos por separado.

Paso 3. Ejecutar una prueba que DEBE fallar

En tu terminal, ejecuta:

```
docker compose exec -T mongo-indices mongosh --quiet --file /lab/06_unique_ttl.js
```

El script:

1. Prepara `unicidad_demo`.
2. Crea un índice único por `email`.
3. Inserta un documento con correo `unico@ejemplo.test`.
4. Intenta insertar otro con el mismo correo.
5. Confirma que MongoDB **rechaza** el segundo intento, en vez de ocultar el error.

Busca el mensaje `E11000` o una confirmación equivalente de que el duplicado fue rechazado.

Interpretación: la primera inserción es correcta. La segunda debe fallar porque contradice la regla que definimos. No es una avería del contenedor.

Paso 4. Explicar la diferencia entre resultado previsto y avería

Anota cuál de estas situaciones demostraría que todo funciona:

- A) Se insertan ambos correos idénticos y el script termina sin error.
- B) Se inserta el primero, se rechaza el segundo y el script confirma el rechazo.
- C) No se inserta ninguno porque la conexión a MongoDB está cortada.

La respuesta es **B**. La situación C no es una prueba de unicidad porque nunca llegamos a comprobar la regla de negocio.

Paso 5. Preparar TTL

El mismo script crea `sesiones_demo` e introduce dos documentos:

- uno cuya fecha `expiraEn` ya pasó;
- otro con una fecha futura.

También crea:

```
db.getSiblingDB("curso_indices").sesiones_demo.createIndex(
  {expiraEn:1},
  {expireAfterSeconds:0}
)
```

¿Por qué cero? Porque el campo almacena directamente **la fecha absoluta de caducidad**. El cero no significa que cualquier sesión desaparezca al insertarse; significa que queda elegible para eliminarse cuando se alcanza su fecha de expiración.

Paso 6. ¿Por qué todavía vemos el registro vencido?

La eliminación no es instantánea: MongoDB utiliza un monitor TTL que funciona periódicamente. Si consultas la colección justo después de crearla y todavía ves el documento vencido, no concluyas inmediatamente que el índice no sirve.

Ejecuta una prueba con margen de espera:

```
docker compose exec -T mongo-indices mongosh --quiet --file /lab/07_ver_ttl.js
```

Ese archivo consulta periódicamente la colección y termina con un error si el registro vencido no desaparece dentro del límite previsto para el laboratorio. También verifica que el registro con fecha futura se conserva.

Resultado esperado: confirmación de que el vencido desapareció y el vigente permanece. Si no lo consigues, revisa que el servidor esté funcionando y que el campo sea de tipo fecha BSON.

Paso 7. Comprobar directamente la colección

```
docker compose exec -T mongo-indices mongosh --quiet --eval
'printjson(db.getSiblingDB("curso_indices").sesiones_demo.find().toArray())'
```

Deberías ver el documento vigente. Escribe qué campos te permiten concluir que no debería estar caducado.

Paso 8. Por qué no debemos confiar en TTL para autorizar una sesión

Imagina que la aplicación comprueba «si el documento existe, la sesión está autorizada». Como el monitor puede tardar en eliminarlo, ese razonamiento es incorrecto: un registro puede existir un breve tiempo aunque su fecha ya haya pasado. La aplicación también debe comprobar la expiración cuando concede acceso.

TTL es un mecanismo de limpieza automática, no una sustitución completa de la lógica de validez de una sesión.

Paso 9. Autocomprobación

1. ¿Por qué la segunda inserción duplicada debe producir un error?
2. Si el documento está caducado a las 10:30, ¿debe desaparecer exactamente a las 10:30:00?
3. ¿Qué ocurriría si `expiraEn` se almacena como una cadena y no como fecha?
4. ¿Cuál es la diferencia entre un índice unique y un índice TTL?

Laboratorio 6 — Consultas cubiertas y mantenimiento de índices

6.1. Nuestro problema ahora es leer menos información

La empresa solicita una lista muy sencilla: identificadores de cliente e importes de sus ventas. No necesita las descripciones, etiquetas, productos ni información de ubicación.

Hasta ahora buscábamos el documento y después examinábamos si cumplía las condiciones. Un índice adecuado puede permitir responder una consulta pequeña **desde sus propias claves**, sin traer el documento completo.

Paso 1. Entender la consulta y su proyección

Con la instancia `mongo-indices` en marcha, abre `mongosh`:

```
docker compose exec mongo-indices mongosh --quiet
```

Dentro de él:

```
const c = db.getSiblingDB("curso_indices").ventas;
c.find(
  {clienteId:809},
  {_id:0,clienteId:1,importe:1}
).limit(3).toArray()
```

El primer objeto es el **filtro**, el segundo es la **proyección**: especifica los campos que queremos ver. `_id:0` excluye expresamente el identificador, que podría impedir una cobertura si el índice no lo contiene.

Paso 2. Crear un índice que contenga lo necesario

```
c.createIndex(
  {clienteId:1,importe:1},
  {name:"ix_cubierta_cliente_importe"}
)
```

Hemos elegido esos dos campos porque aparecen en el filtro y en la proyección. No hemos añadido la descripción ni las etiquetas porque la consulta no las necesita.

Paso 3. Observar el coste

```
const e = c.find(
  {clienteId:809},
  {_id:0,clienteId:1,importe:1}
).hint("ix_cubierta_cliente_importe")
 .limit(3)
 .explain("executionStats");
printjson({
  devueltos:e.executionStats.nReturned,
  documentosExaminados:e.executionStats.totalDocsExamined,
  clavesExaminadas:e.executionStats.totalKeysExamined
});
```

Resultado esperado para esta consulta diseñada como cubierta: `documentosExaminados` puede ser cero. No significa que no haya hecho trabajo; ha recorrido claves del índice.

Paso 4. Romper deliberadamente la cobertura

Ahora pedimos un campo que el índice no contiene:

```
const otro = c.find(
  {clienteId:809},
  {_id:0,clienteId:1,importe:1,descripcion:1}
).hint("ix_cubierta_cliente_importe")
 .limit(3)
 .explain("executionStats");
printjson({
  devueltos:otro.executionStats.nReturned,
  documentosExaminados:otro.executionStats.totalDocsExamined,
  clavesExaminadas:otro.executionStats.totalKeysExamined
});
```

Compara con el paso anterior. Al pedir `descripcion`, MongoDB puede necesitar recuperar documentos para obtener ese dato. Esa diferencia explica el sentido de `FETCH`.

No concluyas que añadir todas las columnas al índice sería siempre mejor. Índices más grandes ocupan espacio y requieren mantenimiento cuando cambian los datos.

Paso 5. Hacer inventario de índices

```
c.getIndexes()
```

Busca nombres, campos y opciones. Un índice puede existir sin que el optimizador lo utilice para la consulta que acabas de ejecutar. Esta lista no sustituye las estadísticas de `explain`.

Paso 6. Ocultar un índice con intención pedagógica

Prueba a ocultar **solo el índice creado por ti en este laboratorio**:

```
c.hideIndex("ix_cubierta_cliente_importe")
```

Si ahora intentas forzar su uso mediante `hint`, el motor puede rechazarlo porque el índice está oculto al planificador. No debemos interpretar eso como que el índice fue eliminado: sigue existiendo y debe mantenerse.

Vuelve a mostrarlo:

```
c.unhideIndex("ix_cubierta_cliente_importe")
```

Comprueba con `getIndexes()` que el estado vuelve a ser visible.

Paso 7. Una retirada controlada

Si has terminado y no vas a reutilizar este índice, puedes eliminar **solo** el que creaste aquí:

```
c.dropIndex("ix_cubierta_cliente_importe")
```

Eso reduce el conjunto de índices disponibles, pero en una base real implicaría analizar qué consultas lo utilizan antes de borrar nada. No elimines `ix_numero` o `ix_esr`: siguen siendo parte del recorrido de aprendizaje.

Paso 8. Tu conclusión

Explica con tus palabras:

- qué significa una proyección;
- por qué excluir `_id` puede resultar importante;
- qué demuestra `totalDocsExamined:0` y qué **no** demuestra;
- por qué ocultar no es lo mismo que eliminar;
- qué riesgos introduce acumular índices indiscriminadamente.

Siguiente desafío: repetir el experimento de índices simples con un millón de ventas, conservando idéntico filtro y registrando las métricas de ambos volúmenes.

Laboratorio 7 — La misma búsqueda con 200.000 y un millón de ventas

Qué queremos demostrar

Hemos dicho que un índice puede evitar que MongoDB examine enormes cantidades de documentos para encontrar uno. ¿Sigue siendo relevante cuando la colección pasa de 200.000 a **1.000.000 de ventas**?

Vamos a responder mediante una experiencia repetible. No vamos a utilizar cronómetros manuales ni proclamar que la consulta es «x veces más rápida». Recogeremos el número real de documentos y claves examinados antes y después de crear un índice.

Este laboratorio **se realiza al final** de las prácticas de índices porque el script de carga borra y recrea la colección `curso_indices.ventas`. No ejecutes la carga si deseas conservar los índices que preparaste para ejercicios anteriores.

Paso 1. Conservar las observaciones anteriores

Antes de modificar datos, copia en tu cuaderno las métricas registradas en el Laboratorio 1 con 200.000 ventas:

Métrica	Sin índice	Con índice
`nReturned`	_____	_____
`totalDocsExamined`	_____	_____
`totalKeysExamined`	_____	_____
`executionTimeMillis`	_____	_____

Si no guardaste esos datos, no los inventes. Puedes repetir la primera práctica completa y anotar las cifras antes de pasar al millón.

Lo importante: registra valores medidos, no valores sugeridos por otra persona. El número 200.000 indica el volumen cargado, no que cualquier plan vaya a examinar exactamente esa cantidad.

Paso 2. Comprobar recursos y servicio

Cargar un millón de documentos requiere más tiempo y espacio que cargar 200.000. Asegúrate de disponer de suficiente almacenamiento y memoria para Docker. Durante el proceso verás mensajes de progreso; no interpretes que una carga de muchos documentos está rota solo porque tarda más que la primera.

Desde tu terminal en la carpeta principal:

```
docker compose up -d --wait mongo-indices
docker compose ps
```

Si el servicio no arranca, consulta `docker compose logs --tail=80 mongo-indices`. No intentes medir consultas hasta que la instancia esté preparada.

Paso 3. Recargar la colección con un millón de documentos

```
docker compose exec -T -e VENTAS=1000000 mongo-indices mongosh --quiet --file /lab/
01_cargar_ventas.js
```

Esta operación es destructiva para la colección de práctica. Elimina `curso_indices.ventas` y la reconstruye con un millón de ventas sintéticas. Si estabas investigando un índice específico, tendrás que volver a crearlo cuando lo necesites.

Mientras se ejecuta, fíjate en los mensajes de avance. No cierres la terminal antes de que el proceso termine o informe de un error.

Paso 4. Verificar que no estamos comparando una carga parcial

```
docker compose exec -T mongo-indices mongosh --quiet --eval
'print(db.getSiblingDB("curso_indices").ventas.countDocuments({}))'
```

Esperamos `1000000`. Si la cifra es menor, revisa el paso de carga. No continúes: el experimento depende del tamaño de la colección.

Comprueba también que no existen índices secundarios:

```
docker compose exec -T mongo-indices mongosh --quiet --eval
'printjson(db.getSiblingDB("curso_indices").ventas.getIndexes())'
```

En una colección recién cargada por nuestro script debe encontrarse únicamente `_id_`. Si ves índices adicionales, la línea base se ha contaminado y el script `02_antes.js` lo señalará.

Paso 5. Repetir la medición SIN índice secundario

```
docker compose exec -T mongo-indices mongosh --quiet --file /lab/02_antes.js
```

El script elige automáticamente una venta próxima al final de la colección. Con un millón de ventas, el valor buscado se ajusta al volumen, de modo que sigue siendo un documento existente.

Anota el plan y los contadores. No confundas que `nReturned` sea 1 con que el motor solo haya examinado 1 documento.

Paso 6. Crear los índices de prácticas

```
docker compose exec -T mongo-indices mongosh --quiet --file /lab/03_crear_indices.js
```

El primer índice por `numero` ayuda a localizar la venta exacta; el script también prepara otros índices de las prácticas. Comprueba que aparece `ix_numero` en el listado.

Pregúntate: ¿por qué crear un índice tarda más sobre un millón de documentos que sobre veinte? Porque el servidor debe procesar un volumen mayor de datos y construir la nueva estructura.

Paso 7. Repetir la medición CON índice

```
docker compose exec -T mongo-indices mongosh --quiet --file /lab/04_despues.js
```

Observa el cambio de plan y de documentos examinados. Para la búsqueda exacta por número, esperamos que un índice útil reduzca de manera drástica los documentos candidatos y que la operación encuentre la misma venta solicitada.

Completa tu segunda tabla:

Métrica	1.000.000 sin índice	1.000.000 con índice
Documentos devueltos	____	____
Documentos examinados	____	____
Claves examinadas	____	____
Plan	____	____
Tiempo orientativo	____	____

Paso 8. Comparar volúmenes de forma razonada

Ahora ya tienes cuatro situaciones:

1. 200.000 documentos, sin índice.
2. 200.000 documentos, con índice.
3. 1.000.000 de documentos, sin índice.
4. 1.000.000 de documentos, con índice.

Responde:

- ¿Cómo cambia el trabajo de un `COLLSCAN` cuando la colección crece?
- ¿Cuántos documentos necesita examinar el caso indexado para una venta identificada de forma unívoca?
- ¿Por qué no debemos extrapolar automáticamente el mismo factor a los milisegundos?
- ¿Qué otros costes habremos añadido al crear índices sobre un millón de ventas?

Solución comentada: cuando el documento buscado está cerca del final y no existe un índice útil, el servidor puede verse obligado a inspeccionar una gran parte de la colección. Un índice selectivo permite dirigirse a pocas claves y documentos, pero también ocupa espacio y requiere trabajo de construcción y mantenimiento.

Paso 9. Detectar un experimento inválido

Un compañero afirma que su índice funciona porque después de crearlo consiguió `nReturned:0` y `totalDocsExamined:0`. ¿Qué puede haber ocurrido?

Respuesta: quizá consultó una venta inexistente. No es correcto celebrar una consulta que no encontró el documento buscado. Primero debemos comprobar que estamos comparando **la misma necesidad de información** y que el resultado es correcto.

Otro compañero compara un filtro por `numero` sin índice con un filtro por `clienteId` tras indexarlo. Tampoco es una comparación limpia: cambió tanto el índice como la pregunta.

Paso 10. Conservar o eliminar los datos

Si vas a seguir trabajando con otras prácticas, conserva el servicio y los datos. Puedes cerrar el shell o detener el contenedor:

```
docker compose stop mongo-indices
```

No ejecutes `down --volumes` por costumbre. Los datos del laboratorio son ficticios, pero debes comprender el efecto de eliminar volúmenes antes de hacerlo.

Laboratorio 8 — Diseñar índices a partir de una necesidad comercial, no de una lista de tipos

El encargo

Una empresa que vende productos en cuatro ciudades te pide mejorar tres pantallas:

- **Pantalla A:** localizar una venta por número cuando llama un cliente.
- **Pantalla B:** mostrar las últimas diez ventas pagadas cuyo importe supera 500 euros.
- **Pantalla C:** mostrar ventas de un cliente, pero únicamente número, fecha e importe.

No nos han pedido «crear tres índices». Nos han pedido **responder tres preguntas de negocio**. Tú deberás justificar qué índice elegirías y cuál podría tener un coste innecesario.

Paso 1. Escribir las consultas antes de crear nada

Dentro de `mongosh` y después de seleccionar `curso_indices`:

```
const c = db.getSiblingDB("curso_indices").ventas;
```

Para A:

```
c.find({numero:199998})
```

Para B:

```
c.find({estado:"pagada",importe:{$gt:500}})
 .sort({fecha:-1})
 .limit(10)
```

Para C:

```
c.find(
  {clienteId:809},
  {_id:0,numero:1,fecha:1,importe:1}
).limit(20)
```

Lee cada comando en voz alta describiendo el filtro, la proyección, la ordenación y el límite. Si no puedes hacerlo, todavía no tiene sentido elegir índices.

Paso 2. Elegir un índice candidato para A

La pantalla A utiliza una igualdad sobre un valor que identifica una venta concreta. El candidato natural es `{numero:1}`, con unicidad si esa es una regla real del dominio.

¿Por qué no incluir otros diez campos? Porque la consulta solo necesita localizar por número; mantener índices grandes puede añadir coste sin aportar valor para esta pregunta.

Paso 3. Elegir un índice candidato para B

La pantalla B combina igualdad de estado, ordenación descendente por fecha y rango de importe. Podemos estudiar:

```
{estado:1,fecha:-1,importe:1}
```

Este es un candidato ESR que trata de conservar la ordenación por fecha dentro de cada estado. También podemos comparar una alternativa ERS:

```
{estado:1,importe:1,fecha:-1}
```

No afirmes que una es la mejor sin medir; la segunda puede reducir claves recorridas en un rango altamente selectivo, pero quizás requiera ordenar los resultados.

Paso 4. Diseñar un índice para C y reflexionar sobre cobertura

La pantalla C filtra por cliente y proyecta número, fecha e importe. Un candidato podría ser:

```
{clienteId:1,numero:1,fecha:1,importe:1}
```

Puede cubrir la consulta si la proyección y las demás condiciones lo permiten. Sin embargo, ¿merece la pena incorporar esos cuatro campos para una pantalla utilizada una vez al día? La respuesta depende de frecuencia, tamaño de los datos y coste de escritura.

Un índice que cubre una consulta no es automáticamente la mejor elección para toda la aplicación.

Paso 5. Elaborar una propuesta defendible

Prepara una tabla:

Pantalla	Índice candidato	Beneficio esperado	Coste o riesgo
A	_____	_____	_____
B	_____	_____	_____
C	_____	_____	_____

Cada fila debe contener una justificación concreta. No basta escribir `IXSCAN` como beneficio: explica qué parte del filtro u ordenación podría utilizarse desde el índice.

Paso 6. Medir en MongoDB

Si ya existen los índices de los laboratorios anteriores, comprueba cuáles están disponibles mediante:

```
c.getIndexes()
```

Luego ejecuta cada consulta con `explain("executionStats")`. Observa planes, documentos devueltos, claves examinadas, documentos examinados y etapas de ordenación.

Si todavía no existe un índice candidato, puedes crearlo **en el entorno de laboratorio**, ejecutar la consulta y comparar su plan. Documenta siempre qué índice acabas de crear para poder distinguir lo anterior de lo nuevo.

Paso 7. Presentar conclusiones comprensibles

No entregues un JSON enorme sin explicación. Prepara un resumen para la persona responsable de la empresa:

- «La pantalla A busca una venta concreta. Un índice por número permite localizarla sin recorrer toda la colección».
- «La pantalla B es más compleja porque además de filtrar debe ordenar. Hemos medido dos estrategias y observado el trabajo de cada una».
- «La pantalla C solo necesita algunos campos. Una proyección adecuada puede evitar transferir datos innecesarios y un índice estudiado podría cubrir la consulta».

Evaluación: si alguien que no conoce `mongosh` puede entender qué problema resolviste, has completado la parte profesional del ejercicio. Si solo ve una lista de comandos, todavía falta tu explicación.

Laboratorio 9 — Unicidad condicionada: cupones activos e índices parciales

La historia de negocio

Una tienda emite códigos promocionales. Quiere impedir que existan **dos promociones activas con el mismo código**. Sin embargo, un cupón de una campaña anterior puede estar inactivo y su código podría reutilizarse cuando la política comercial lo permita.

La regla correcta no es «todos los códigos de toda la historia deben ser únicos». Es una regla **condicionada por el estado activo**.

Este laboratorio enseña a traducir una frase de negocio en un índice `unique` con una condición `partialFilterExpression`. También enseña que una actualización puede infringir la restricción aunque la inserción original fuera válida.

Paso 1. Crear una colección independiente

Abre `mongosh` en la instancia de índices:

```
docker compose exec mongo-indices mongosh --quiet
```

Dentro del shell:

```
const d = db.getSiblingDB("curso_indices");
d.cupones_clase.drop();
```

La orden `drop` elimina **solo** la colección ficticia `cupones_clase`. No modifica las ventas principales.

Paso 2. Formular la restricción

Queremos que el campo `codigo` sea único **solo cuando `activo` sea verdadero**. Creamos:

```
d.cupones_clase.createIndex(
  {codigo:1},
  {
    name:"ix_codigo_activo",
    unique:true,
    partialFilterExpression:{activo:true}
  }
);
```

Piensa en las piezas:

- `codigo:1` define la clave del índice.
- `unique:true` prohíbe duplicar claves dentro del conjunto afectado.
- `partialFilterExpression` determina qué documentos pertenecen a ese conjunto.

No estamos diciendo que `activo` sea parte de la clave ordenada; lo usamos como **condición de pertenencia al índice**.

Paso 3. Introducir un cupón activo

```
d.cupones_clase.insertOne({
  _id:1,
  codigo:"PROMO7",
  activo:true,
  descripcion:"Campaña actual"
});
```

Debe insertarse correctamente. El índice incorpora el valor `PROMO7` porque el documento cumple el filtro `activo:true`.

Paso 4. Introducir un cupón inactivo con el mismo código

```
d.cupones_clase.insertOne({
  _id:2,
  codigo:"PROMO7",
  activo:false,
  descripcion:"Campaña antigua"
});
```

Este documento **no cumple el filtro parcial** y, bajo la regla que acabamos de definir, no entra en el conjunto de claves cuya unicidad controlamos. Se espera una inserción permitida.

Comprueba el estado de la colección:

```
d.cupones_clase.find({codigo:"PROMO7"}).toArray()
```

Deben verse dos cupones con el mismo código, uno activo y otro inactivo. No es una contradicción: el requisito no prohibía todos los duplicados históricos.

Paso 5. Intentar activar el duplicado

```
d.cupones_clase.updateOne(
  {_id:2},
  {$set:{activo:true}}
);
```

Resultado esperado: rechazo por violación de unicidad. La actualización haría que el segundo cupón entrara en el conjunto indexado donde ya existe otro activo con el mismo código.

Este paso muestra una propiedad fundamental: **los índices afectan también a las actualizaciones**. Una restricción sobre datos de negocio no se limita al momento de insertar.

Paso 6. Comprobar que el documento sigue inactivo

```
d.cupones_clase.findOne({_id:2})
```

El valor `activo` debe continuar siendo falso si la actualización se rechazó. Así verificamos que el rechazo protegió la integridad de los documentos.

Paso 7. Ensayar la reutilización después de desactivar el original

Desactiva el primer cupón:

```
d.cupones_clase.updateOne(
  {_id:1},
  {$set:{activo:false}}
);
```

Ahora intenta activar el segundo:

```
d.cupones_clase.updateOne(
  {_id:2},
  {$set:{activo:true}}
);
```

Resultado esperado: esta vez sí puede activarse, porque ya no existe otro cupón activo con `PROMO7`. Consulta ambos documentos y comprueba que la regla final se cumple.

Paso 8. Explicar la diferencia entre índice simple, único y parcial

Diseño	Qué resolvería	Limitación respecto al negocio
`{codigo:1}` normal	Facilitar búsquedas por código	No impide duplicados
`{codigo:1}` unique general	Impedir duplicados en toda la colección	Puede ser demasiado restrictivo para cupones antiguos
Unique con filtro `activo:true`	Impedir coincidencias entre cupones activos	Depende de que `activo` esté modelado correctamente

Debate: ¿qué debe suceder si un documento no tiene campo `activo`? No basta con dejar que la aplicación «lo interprete como quiera». Hay que definir una política coherente para ese estado.

Paso 9. Comprobar de forma automática

El repositorio contiene un archivo de prueba complementario, `lab/17_partial_unique_guiado.js`, que ejecuta el escenario con datos de ejemplo y comprueba los rechazos esperados. Puedes ejecutarlo en una colección aislada desde la terminal:

```
docker compose exec -T mongo-indices mongosh --quiet --file /lab/17_partial_unique_guiado.js
```

No ejecutes este script antes de guardar tus observaciones manuales: utiliza su propia colección de prueba y la recrea para garantizar un punto de partida conocido.

Evaluación

Una alumna propone crear `{codigo:1,activo:1}` con `unique:true` y afirma que así resuelve exactamente el problema. ¿Es equivalente a nuestro diseño parcial?

Piénsalo: un índice único compuesto sobre esas dos claves puede restringir también que existan dos cupones **inactivos** con el mismo código. La regla de negocio solo exigía unicidad entre los activos. Comprender esta diferencia es más importante que recordar de memoria la palabra `partialFilterExpression`.

Laboratorio 10 — Investigar un índice que nadie utiliza

El encargo

Un administrador dice: «He creado el índice `ix_cliente` y la búsqueda sigue comportándose como antes. ¿Está MongoDB ignorando mi trabajo?». No vamos a responder que el índice está mal sin observar la consulta. Puede ser un índice poco útil, una pregunta distinta de la que motivó su creación, un plan de consulta razonable o una colección demasiado pequeña.

Paso 1. Identificar la consulta real

Supongamos que la pantalla pide ventas por `estado`, no por `clienteId`:

```
db.ventas.find({estado:"pagada"}).limit(100)
```

Aunque exista `ix_cliente`, este índice está organizado por otro campo. Su mera existencia no garantiza utilidad para esta consulta.

Paso 2. Consultar índices reales

Dentro de `mongosh`:

```
const c = db.getSiblingDB("curso_indices").ventas;
c.getIndexes()
```

Busca:

- nombre del índice;
- campos y direcciones;
- propiedades como `unique` u opciones adicionales.

No deduzcas que los índices con nombres largos son mejores. El nombre sirve para identificarlos; son su clave y opciones las que determinan su función.

Paso 3. Inspeccionar el plan sin forzar nada

```
c.find({estado:"pagada"}).limit(100).explain("executionStats")
```

Observa el plan ganador y los contadores. Si devuelve cien documentos, no significa automáticamente que examinará cien; podría necesitar recorrer más claves o documentos para localizar esos cien.

Paso 4. Formular una hipótesis

Si el optimizador no elige `ix_cliente`, una hipótesis razonable es que la consulta no contiene un filtro que aproveche su ordenación. Si elige otro índice, quizá este sirva mejor a los criterios consultados.

No debes cambiar código hasta expresar qué esperabas del índice y por qué.

Paso 5. Comprobar con una consulta del campo indexado

```
c.find({clienteId:809}).limit(10).explain("executionStats")
```

Compara plan y contadores. La nueva consulta ya pregunta por el campo para el que se creó `ix_cliente`. **No compares los tiempos de estas dos consultas como si fueran antes/después del mismo experimento**: acabas de cambiar el filtro. Utilízalas únicamente para mostrar que un índice está vinculado a necesidades concretas.

Paso 6. Ensayar una retirada sin destruir el índice

En una colección de laboratorio puedes ocultar temporalmente `ix_cliente`:

```
c.hideIndex("ix_cliente")
```

Repite la consulta por cliente, pero **sin `hint`**, para comprobar cómo planifica MongoDB sin poder utilizar ese índice oculto. Después restáuralo:

```
c.unhideIndex("ix_cliente")
```

Ocultar no reduce automáticamente el espacio ocupado ni el mantenimiento de ese índice; es un mecanismo de estudio de planes y gestión.

Paso 7. Registrar una decisión argumentada

Completa:

- Consulta que motivó el índice: _____.
- Índice realmente definido: _____.
- Plan sin forzar: _____.
- Documentos y claves examinados: _____.
- ¿La consulta utiliza un campo compatible con el índice?: _____.
- ¿Qué otras consultas podrían necesitarlo?: _____.

Conclusión: decidir si eliminar un índice exige comprender el conjunto de consultas de la aplicación, no una única medición. En producción los cambios deben estudiarse con datos de uso, pruebas y procedimientos de recuperación.

Laboratorio 11 — Diagnosticar una consulta que devuelve cero ventas

El error que vamos a investigar

Una persona afirma que el índice de ventas «ha dejado de funcionar» porque su consulta devuelve cero resultados. Lo primero que debes aprender es que **cero resultados no demuestra que el índice esté roto**. Una consulta puede ser perfectamente correcta, ejecutarse sin errores y devolver cero simplemente porque no existe un documento que cumpla su filtro.

Vamos a distinguir cuatro causas que, desde una pantalla de usuario, pueden parecer iguales: filtro incorrecto, campo mal escrito, base de datos equivocada y documento realmente inexistente.

Preparación

Este ejercicio **no requiere borrar ni recargar las ventas**. Utiliza las que ya están almacenadas en `curso_indices.ventas`. Abre una terminal en la raíz del proyecto:

```
docker compose up -d --wait mongo-indices
docker compose exec mongo-indices mongosh --quiet
```

La primera línea se ejecuta en PowerShell o Bash. La segunda abre `mongosh` dentro del contenedor. A partir de ahora los ejemplos JavaScript se escriben **dentro de `mongosh`**.

Paso 1. Comprobar si existen datos

```
const ventas = db.getSiblingDB("curso_indices").ventas;
ventas.countDocuments({})
```

Debes obtener una cifra positiva. Si aparece cero, no continúes buscando un fallo de índices: aún no has confirmado que existe el conjunto de datos. Realiza la carga del primer laboratorio antes de seguir.

Paso 2. Buscar un número existente

```
ventas.findOne({numero:1})
```

El generador docente crea números consecutivos desde uno. Por tanto, debería aparecer un documento cuyo campo `numero` sea uno. Observa su `estado`, `importe` y `clienteId`.

Anota los tres valores reales. No copies los de este texto: vamos a trabajar sobre lo que contenga tu base.

Paso 3. Buscar un número inexistente

```
ventas.findOne({numero:-999})
```

Este filtro busca un número que no forma parte de la secuencia de ventas generada. Si aparece `null` o no se encuentra un documento, la explicación es sencilla: no existe una venta con ese identificador. No hay un error de conexión ni una violación de la integridad.

Paso 4. Introducir un nombre de campo equivocado

```
ventas.findOne({número:1})
```

Fíjate en la tilde: el generador creó `numero` sin tilde. MongoDB no interpreta ambos nombres como sinónimos. **No se puede solucionar esta diferencia creando un índice de forma indiscriminada**: primero hay que corregir el filtro.

Si obtienes cero resultados, explica por qué esta situación es distinta de buscar el número -999: ahora la base sí contiene el identificador uno, pero hemos escrito mal el campo.

Paso 5. Consultar otra base de datos

Compara:

```
db.getSiblingDB("curso_indices").ventas.findOne({numero:1})
db.getSiblingDB("otra_base").ventas.findOne({numero:1})
```

En el laboratorio solo preparamos las ventas dentro de `curso_indices`. Si en `otra_base` no existe esa colección, la consulta no encontrará el documento. Esto demuestra que **el nombre de la base también forma parte del contexto de cada operación**.

No introduzcas datos en `otra_base` para «arreglar» el resultado: la pregunta correcta se refiere a `curso_indices`.

Paso 6. Comparar un filtro de igualdad con un rango

```
ventas.find({importe:500}).limit(3).toArray()
ventas.find({importe:{$gt:500}}).limit(3).toArray()
```

La primera pide exactamente 500. La segunda pide cualquier importe superior a 500. Pueden devolver cantidades muy distintas, incluso si hay un índice por importe. El índice no cambia el significado del símbolo `$gt`.

Paso 7. Verificar sin confundir la consulta con el plan

Para una búsqueda que encuentra una venta:

```
const e = ventas.find({numero:1}).explain("executionStats");
printjson({
  encontrados:e.executionStats.nReturned,
  examinados:e.executionStats.totalDocsExamined,
  claves:e.executionStats.totalKeysExamined
});
```

Ahora repite con `numero:-999`. Es posible que un índice localice rápidamente que la clave no existe y examine muy poco. Ese resultado no demuestra que la segunda consulta «sea mejor»: las preguntas tienen resultados distintos.

Paso 8. Redactar un diagnóstico profesional

Completa una tabla con **tus propias observaciones**:

Consulta	Resultado esperado	Causa si devuelve cero
`numero:1`	Existe	Revisar carga o base de datos
`numero:-999`	No existe	Valor ausente
`número:1`	No existe	Campo escrito de otro modo
Otra base de datos	No existe	Contexto equivocado

Explica por qué cada caso requiere una corrección diferente. El aprendizaje clave es que **optimizar una consulta incorrecta no convierte su resultado en correcto**.

Laboratorio 12 — Orden estable y paginación: el problema de las fechas empatadas

Una pantalla puede mostrar diez resultados distintos sin que se haya perdido ninguno

La empresa quiere mostrar las diez ventas pagadas más recientes. Has aprendido a escribir `sort({fecha:-1}).limit(10)`. Pero ahora aparece una incidencia: al repetir la pantalla, en ocasiones los números de venta en la última posición no son los mismos.

¿Se ha perdido información? No necesariamente. Si varios documentos comparten fecha, ordenar únicamente por fecha no define **qué documento va primero dentro del empate**. Cuando además utilizamos un límite de diez, los empates cercanos al corte pueden influir en qué documentos aparecen.

Este ejercicio enseña algo imprescindible en aplicaciones reales: **ordenar por un campo que puede repetirse no garantiza un orden total**.

Paso 1. Crear una colección de solo seis documentos

Usaremos una colección de clase independiente para comprender el problema sin depender de 200.000 ventas. Dentro de `mongosh`:

```
const d = db.getSiblingDB("curso_indices");
d.orden_empates_clase.drop();
d.orden_empates_clase.insertMany([
  {_id:1,estado:"pagada",fecha:ISODate("2025-05-01"),importe:90},
  {_id:2,estado:"pagada",fecha:ISODate("2025-05-03"),importe:110},
  {_id:3,estado:"pagada",fecha:ISODate("2025-05-03"),importe:120},
  {_id:4,estado:"pagada",fecha:ISODate("2025-05-03"),importe:130},
  {_id:5,estado:"pagada",fecha:ISODate("2025-05-02"),importe:140},
  {_id:6,estado:"pendiente",fecha:ISODate("2025-05-04"),importe:150}
]);
```

El `drop` solo elimina `orden_empates_clase`, una colección de prueba. Las ventas principales no se modifican.

Paso 2. Ordenar exclusivamente por fecha

```
d.orden_empates_clase.find(
  {estado:"pagada"}
).sort({fecha:-1}).limit(2).toArray()
```

Los documentos del día 3 deben aparecer antes que los del día 2. Pero existen **tres ventas pagadas del día 3**, todas empatadas en `fecha`. Con un límite de dos, ¿cuáles deben aparecer? El filtro y la ordenación no especifican cómo resolver ese empate.

No interpretes que falta un documento de la colección: limitaste explícitamente los resultados a dos. Los demás continúan almacenados.

Paso 3. Crear un orden total

Añade el identificador como criterio de desempate:

```
d.orden_empates_clase.find(
  {estado:"pagada"}
).sort({fecha:-1,_id:1}).limit(2).toArray()
```

Ahora el orden lógico es:

1. Primero la fecha descendente.
2. Para fechas idénticas, identificador ascendente.

Resultado esperado: aparecen los documentos del día 3 con identificadores 2 y 3. Ya no hemos dejado el empate sin resolver.

Paso 4. Comparar índices candidatos

```
d.orden_empates_clase.createIndex(
  {estado:1,fecha:-1},
  {name:"ix_sin_desempate"}
);
d.orden_empates_clase.createIndex(
  {estado:1,fecha:-1,_id:1},
  {name:"ix_con_desempate"}
);
```

El primer índice ordena por estado y fecha, pero no contiene como tercera clave explícita el desempate que solicita la consulta. El segundo sí incorpora los tres campos que estamos estudiando.

No concluyas automáticamente cuál elegirá el optimizador. Usa `explain` para observarlo; si quieres comparar, emplea `hint` de forma controlada sobre una consulta idéntica.

Paso 5. Visualizar el plan

```
d.orden_empates_clase.find({estado:"pagada"})
  .sort({fecha:-1,_id:1})
  .limit(2)
  .hint("ix_con_desempate")
  .explain("executionStats")
```

Observa el plan, las claves examinadas y el resultado. Con seis documentos el tiempo será poco significativo. La práctica es de **semántica y diseño de índices**, no una carrera de milisegundos.

Paso 6. Paginación con skip y limit

Podemos pedir «la primera página» de dos documentos y la segunda página de dos:

```
d.orden_empates_clase.find({estado:"pagada"})
  .sort({fecha:-1,_id:1}).skip(0).limit(2).toArray()
d.orden_empates_clase.find({estado:"pagada"})
  .sort({fecha:-1,_id:1}).skip(2).limit(2).toArray()
```

`skip` indica cuántos resultados se omiten después de aplicar el orden. En esta colección pequeña es útil para aprender. Sobre colecciones grandes, omitir muchos resultados puede implicar recorrerlos, por lo que la paginación por desplazamiento puede encarecerse.

Paso 7. Comprender la paginación por cursor

Cuando se ordena por fecha e identificador, es posible diseñar una paginación basada en el último par de valores visto, en vez de omitir miles de documentos. Esto requiere expresar correctamente una condición de continuación coherente con las direcciones de ordenación.

No vamos a convertir un ejemplo introductorio en una arquitectura de paginación profesional completa, pero **sí debemos comprender por qué el desempate es imprescindible** para un cursor estable.

Paso 8. Comprobar que todo sigue en la colección

```
d.orden_empates_clase.countDocuments({})
```

Debe devolver seis. No confundas cantidad almacenada y cantidad mostrada por una página.

Actividad final

Explica por qué estos dos fragmentos expresan pedidos diferentes:

```
.sort({fecha:-1}).limit(2)
.sort({fecha:-1,_id:1}).limit(2)
```

Tu respuesta debe mencionar los empates, el criterio secundario y el hecho de que el límite actúa sobre una secuencia ordenada. La aplicación no puede asumir que MongoDB adivinará el orden de negocio que nadie especificó.

Laboratorio 13 — ¿Qué ocurre al modificar una clave de índice?

Un índice no es una fotografía congelada

Hasta ahora hemos creado índices y ejecutado consultas. Pero una base de datos real cambia: se insertan ventas, se actualizan sus estados y se corrigen importes. MongoDB debe mantener los índices coherentes con esos cambios.

Si una aplicación modifica una venta de `pendiente` a `pagada`, un índice por `estado` no puede seguir tratando el documento como pendiente. La actualización de los datos debe reflejarse en las estructuras de acceso.

Vamos a observarlo en una colección diminuta que podemos modificar sin riesgo.

Paso 1. Preparar los datos

Dentro de `mongosh` conectado a la instancia `mongo-indices`:

```
const d = db.getSiblingDB("curso_indices");
d.cambios_indice_clase.drop();
d.cambios_indice_clase.insertMany([
  {_id:1,estado:"pendiente",importe:45},
  {_id:2,estado:"pagada",importe:70},
  {_id:3,estado:"pagada",importe:80}
]);
```

Comprueba que hay una pendiente y dos pagadas:

```
d.cambios_indice_clase.countDocuments({estado:"pendiente"})
d.cambios_indice_clase.countDocuments({estado:"pagada"})
```

Esperamos uno y dos, respectivamente.

Paso 2. Crear un índice compuesto

```
d.cambios_indice_clase.createIndex(
  {estado:1,importe:1},
  {name:"ix_estado_importe"}
);
```

Ahora el índice organiza las ventas primero por estado y, dentro de cada estado, por importe. Si la aplicación busca todas las pagadas, puede beneficiarse de ese índice según el plan elegido.

Paso 3. Modificar el estado

```
d.cambios_indice_clase.updateOne(
  {_id:1},
  {$set:{estado:"pagada"}}
);
```

Resultado esperado: una actualización reconocida. La venta identificada con uno ya no pertenece lógicamente al conjunto de pendientes.

Vuelve a contar:

```
d.cambios_indice_clase.countDocuments({estado:"pendiente"})
d.cambios_indice_clase.countDocuments({estado:"pagada"})
```

Ahora esperamos cero pendientes y tres pagadas.

Paso 4. Forzar el índice como instrumento de estudio

```
d.cambios_indice_clase.find({estado:"pagada"})
 .hint("ix_estado_importe").toArray()
```

Deben aparecer los tres documentos. Esto ilustra que el índice refleja la información actual, no la que tenía el documento cuando creamos el índice.

Paso 5. Actualizar el importe

```
d.cambios_indice_clase.updateOne(
  {_id:2},
  {$set:{importe:10}}
);
```

Consulta las pagadas ordenadas por importe:

```
d.cambios_indice_clase.find({estado:"pagada"})
 .sort({importe:1})
 .hint("ix_estado_importe").toArray()
```

La venta dos debe aparecer con importe diez. Los índices no «guardan el valor antiguo» como si fueran un archivo de consulta manual que nunca se actualiza.

Paso 6. Comprender el coste de mantenimiento

Cada inserción, actualización o borrado que afecta a campos indexados puede requerir modificar las entradas correspondientes. Por eso los índices tienen un coste de escritura.

Pregunta profesional: ¿por qué no deberíamos crear un índice compuesto enorme que contenga todos los campos de una venta solo para evitar algunas recuperaciones de documentos?

Respuesta orientativa: porque aumenta el tamaño y el trabajo de mantenerlo. Hay que demostrar que las consultas justifican ese coste.

Paso 7. Probar una restricción de integridad

Crea un índice único sobre un campo de ejemplo en una colección aislada:

```
d.codigo_unico_clase.drop();
d.codigo_unico_clase.createIndex({codigo:1},{unique:true});
d.codigo_unico_clase.insertMany([
  {_id:1,codigo:"A"},
  {_id:2,codigo:"B"}
]);
```

Ahora intenta actualizar el código B por A:

```
d.codigo_unico_clase.updateOne({_id:2},{$set:{codigo:"A"}})
```

Resultado esperado: un error de clave duplicada. Eso demuestra que una restricción unique no se aplica únicamente al insertar: también protege actualizaciones.

Paso 8. Confirmar que el dato original permanece

```
d.codigo_unico_clase.findOne({_id:2})
```

El código debe seguir siendo B. Si estuvieras escribiendo una aplicación, tendrías que decidir cómo comunicar al usuario que su actualización infringió una regla.

Preguntas de comprensión

- ¿Qué observación demuestra que el índice sigue representando los datos actuales?
- ¿Por qué el índice unique debe comprobar también una actualización?
- ¿Qué relación existe entre más índices y mayor trabajo de escritura?
- ¿Por qué una restricción de negocio no debería implementarse únicamente con una comprobación manual en la pantalla de la aplicación?

Laboratorio 14 — Elaborar un informe de rendimiento que otra persona pueda entender

Una prueba no sirve de mucho si nadie puede interpretarla

Imagina que tu responsable te pide «demuéstrame que el nuevo índice merece la pena». No bastará con enviar un archivo JSON de `explain` de mil líneas ni con afirmar «en mi ordenador funcionaba rápido». Hay que construir una explicación que relacione una consulta real, los datos disponibles, el plan y los resultados.

Este laboratorio no exige crear índices nuevos. **Reutiliza resultados observados** y te enseña a redactar una conclusión técnica rigurosa.

Paso 1. Identificar la necesidad

Describe la pantalla o proceso que estudiaste. Ejemplo:

> «Una aplicación de atención al cliente necesita encontrar rápidamente una venta por su número».

Es mucho mejor que escribir «optimización del índice ix_numero», porque primero explica el propósito del sistema.

Paso 2. Documentar el filtro exacto

Por ejemplo:

```
db.getSiblingDB("curso_indices").ventas.find({numero:199998})
```

Aclara que el número concreto debe existir en el conjunto cargado y que la consulta devuelve una venta identificada de forma inequívoca.

Paso 3. Registrar el volumen y el estado de los índices

```
const c = db.getSiblingDB("curso_indices").ventas;
c.countDocuments({})
c.getIndexes()
```

Escribe cuántos documentos había, qué índices relevantes existían y si la medición procede de una colección recién cargada o de un entorno con varios índices de prácticas anteriores.

No mezcles métricas de estados distintos. Si recargaste la colección, posiblemente cambió el catálogo de índices.

Paso 4. Escoger métricas con significado

Un resumen útil:

Dato	Sin índice	Con índice
Documentos devueltos	Medido	Medido
Documentos examinados	Medido	Medido
Claves examinadas	Medido	Medido
Etapas principales	Observadas	Observadas
Tiempo de esa ejecución	Observado	Observado

Cuando no dispongas del estado «sin índice» porque ya creaste estructuras, no lo inventes. Repite el experimento en el laboratorio o explica que solo tienes evidencia parcial.

Paso 5. Comprobar si la comparación es justa

Pregunta:

- ¿La consulta era la misma?
- ¿La cantidad de datos era la misma?
- ¿La proyección y el límite coincidían?
- ¿El documento existía en ambas mediciones?
- ¿Sabemos qué índice se utilizó?
- ¿Se ejecutó en un entorno comparable?

Si alguna respuesta es no, el informe debe indicar esa limitación. La honestidad de la medición vale más que una cifra espectacular.

Paso 6. Explicar la diferencia entre hechos e inferencias

Hecho observado: «En una ejecución, MongoDB examinó 200.000 documentos sin índice y uno después de crear un índice por número».

Interpretación razonable: «En esta búsqueda muy selectiva, el índice redujo el trabajo necesario para localizar el documento».

Afirmación no respaldada: «Cualquier consulta será 200.000 veces más rápida».

Separar estas frases es parte de aprender a evaluar un sistema.

Paso 7. Evaluar costes y límites

Tu informe debe mencionar al menos uno de los siguientes aspectos:

- almacenamiento ocupado por el índice;
- mantenimiento ante inserciones y actualizaciones;
- impacto de cambios en el volumen de datos;
- diferencia entre consultas selectivas y consultas que devuelven muchos documentos;
- variabilidad de los tiempos observados.

Paso 8. Presentación final

Escribe un informe de una página con:

1. Problema original.
2. Consulta analizada.
3. Conjunto de datos.
4. Índice elegido y justificación.
5. Evidencia antes/después.
6. Costes y limitaciones.
7. Próxima comprobación que harías antes de producción.

Una persona ajena a MongoDB debería entender **por qué el índice importa** sin tener que leer todo el JSON original.

Solución de ejemplo

«La pantalla busca una venta concreta entre 200.000 registros. Sin un índice secundario por número, la consulta necesitó examinar una gran cantidad de documentos para devolver uno. Después se añadió un índice ascendente único sobre el número. Al repetir el mismo filtro, el plan utilizó el índice y los documentos examinados se redujeron drásticamente. Esta mejora se refiere al trabajo de la consulta medida; los tiempos son específicos del equipo y el índice introduce un coste de almacenamiento y mantenimiento».

Ese informe sí demuestra comprensión. Una captura del mensaje `[OK]`, por sí sola, no.

Laboratorio 15 — Búsquedas sin distinguir mayúsculas: collation e índices

Por qué «Ana» y «ana» no siempre deben ser personas distintas

Una aplicación de inscripciones permite buscar alumnos por su nombre. Un usuario escribe «ana» y espera encontrar a «Ana». En otra aplicación, un código de producto `AB-10` debe tratarse como diferente de `ab-10` por una regla del negocio.

Antes de indexar hay que decidir **qué significa considerar dos valores iguales**. Los índices no pueden tomar esa decisión por nosotros.

MongoDB permite definir reglas de comparación lingüística mediante **collation**. Una collation puede influir en cómo se comparan y ordenan cadenas, incluido si distinguimos mayúsculas, acentos y otros aspectos según idioma y fuerza de comparación.

Vamos a estudiar un caso sencillo con nombres ficticios.

Paso 1. Iniciar MongoDB y abrir mongosh

Desde PowerShell o Bash:

```
docker compose up -d --wait mongo-indices
docker compose exec mongo-indices mongosh --quiet
```

Las instrucciones siguientes se ejecutan **dentro de `mongosh`**. No vamos a tocar `ventas`.

Paso 2. Crear una colección de nombres

```
const d = db.getSiblingDB("curso_indices");
d.nombres_clase.drop();
d.nombres_clase.insertMany([
  {_id:1,nombre:"Ana",ciudad:"Madrid"},
  {_id:2,nombre:"Carlos",ciudad:"Sevilla"},
  {_id:3,nombre:"Beatriz",ciudad:"Bilbao"}
]);
```

El `drop` afecta únicamente a la colección de clase `nombres_clase`. Observa que el nombre del primer documento empieza por mayúscula.

Paso 3. Consultar por un valor exacto sin collation explícita

```
d.nombres_clase.find({nombre:"ana"}).toArray()
```

Según las reglas de comparación por defecto de la colección, no debemos suponer que `"ana"` y `"Ana"` son iguales. En este escenario esperamos no encontrar el documento cuyo nombre fue almacenado con mayúscula.

Paso 4. Aplicar una comparación lingüística

```
d.nombres_clase.find({nombre:"ana"})
 .collation({locale:"es",strength:2})
 .toArray()
```

La opción `locale:"es"` utiliza reglas lingüísticas de español; `strength:2` permite una comparación que no distingue mayúsculas de minúsculas en este caso, manteniendo otras distinciones lingüísticas propias de la configuración.

Resultado esperado: ahora podemos encontrar el documento `Ana`. No se modificó lo almacenado: cambiamos la regla de comparación aplicada a la consulta.

Paso 5. Crear un índice con esa misma collation

```
d.nombres_clase.createIndex(
  {nombre:1},
  {
    name:"ix_nombre_es",
    collation:{locale:"es",strength:2}
  }
)
```

El índice está construido de acuerdo con unas reglas lingüísticas concretas. Eso implica una idea importante: **no cualquier consulta podrá aprovecharlo del mismo modo si utiliza una collation diferente**.

Paso 6. Comprobar el uso del índice

```
d.nombres_clase.find({nombre:"ana"})
 .collation({locale:"es",strength:2})
 .hint("ix_nombre_es")
 .explain("executionStats")
```

Busca en la salida si se utiliza el índice y cuántas claves se examinan. La colección contiene solo tres documentos, por lo que no esperamos demostrar una gran mejora temporal. Esta actividad trata de **semántica de comparación** y de compatibilidad entre consulta e índice.

Paso 7. Una restricción unique con comparación lingüística

Imagina que dos registros no deben distinguirse únicamente por mayúsculas. En una colección independiente:

```
d.nombres_unicos_clase.drop();
d.nombres_unicos_clase.createIndex(
  {nombre:1},
  {
    name:"ix_nombre_unico_es",
    unique:true,
    collation:{locale:"es",strength:2}
  }
);
d.nombres_unicos_clase.insertOne({nombre:"Ana"});
```

Ahora intenta insertar un segundo documento con `nombre:"ana"`. **Resultado esperado:** la operación debe rechazarse por duplicado según la collation del índice único.

La prueba es valiosa porque enseña que una regla de comparación también puede afectar a la integridad.

Paso 8. Explicar sin engaños

No afirmes que `strength:2` sirve para resolver cualquier necesidad lingüística de cualquier idioma. Las reglas de mayúsculas, acentos, caracteres equivalentes y ordenación deben estudiarse en el contexto real.

Para este ejercicio basta con argumentar:

- por qué la consulta inicial no encontraba la cadena con mayúscula;
- qué cambió al declarar la collation;
- por qué el índice debe estar alineado con las reglas de comparación;
- por qué un índice unique puede rechazar valores que un usuario considera equivalentes.

Pregunta de diseño

Una empresa utiliza identificadores de producto que distinguen mayúsculas, pero los nombres de sus clientes no. ¿Aplicarías necesariamente la misma collation a ambos campos? Justifica la respuesta a partir de las reglas del negocio, no de una preferencia estética.

Laboratorio 16 — Una búsqueda de subcadenas no es un índice de texto

El problema

En el Laboratorio 3 utilizaste `$text` para buscar términos dentro de descripciones. Ahora otro compañero pide un buscador que encuentre cualquier fragmento escrito en medio de una palabra. Afirma que «si la base tiene un índice de texto, todas las búsquedas por letras deberían ser rápidas».

Esta afirmación es incorrecta: **buscar palabras mediante un índice de texto clásico** y **buscar una secuencia arbitraria de caracteres** son problemas con semánticas y costes diferentes.

Paso 1. Preparar tres descripciones

Utiliza una colección ficticia, sin alterar `ventas`:

```
const d = db.getSiblingDB("curso_indices");
d.descripcion_clase.drop();
d.descripcion_clase.insertMany([
  {_id:1,descripcion:"cafe premium en grano"},
  {_id:2,descripcion:"cafetera para oficina"},
  {_id:3,descripcion:"bicicleta urbana plegable"}
]);
```

Piensa qué debería encontrar un usuario si escribe `cafe`, `cafet` o `premium`. ¿Consideramos coincidencia de palabra, prefijo de palabra o subcadena arbitraria?

Paso 2. Buscar por igualdad completa

```
d.descripcion_clase.find({descripcion:"cafe"}).toArray()
```

Resultado esperado: no encuentra el documento uno, porque su descripción completa no es exactamente `cafe`. El filtro de igualdad compara el valor completo.

Paso 3. Buscar una secuencia mediante regex

```
d.descripcion_clase.find({
  descripcion:{$regex:"cafe",$options:"i"}
}).toArray()
```

Ahora pedimos una expresión regular que busque la secuencia `cafe`, ignorando el caso de las letras. En estos datos puede localizar tanto `cafe premium` como `cafetera`.

Eso no significa que la consulta vaya a aprovechar automáticamente un índice `text`. Una expresión regular no equivale a la búsqueda textual por términos.

Paso 4. Crear el índice textual

```
d.descripcion_clase.createIndex(
  {descripcion:"text"},
  {name:"ix_texto_clase",default_language:"spanish"}
);
```

El índice habilita las consultas mediante `$text`. No convierte todas las expresiones regulares en búsquedas optimizadas.

Paso 5. Ejecutar una consulta textual

```
d.descripcion_clase.find({
  $text:{$search:"premium"}
}).toArray()
```

Resultado esperado: el documento que contiene `cafe premium en grano`. La operación busca el término de acuerdo con el índice de texto disponible.

Paso 6. Comparar planes sin falsear la equivalencia

Puedes llamar a `explain("executionStats")` sobre la consulta regex y sobre la consulta `$text`. Pero **no las presentes como un antes/después del mismo problema**: las reglas de coincidencia son diferentes y, por tanto, la cantidad de resultados también puede cambiar.

El objetivo es comprender:

- qué resultado pide cada filtro;
- qué estructura permite resolverlo;
- cuándo una técnica no sustituye a la otra;
- por qué necesitamos elegir el buscador a partir de requisitos reales.

Paso 7. Precauciones frente a sistemas de búsqueda más avanzados

Un buscador corporativo puede necesitar corrección ortográfica, relevancia compleja, tolerancia a errores, sinónimos y análisis de idiomas. Los índices `text` clásicos permiten ciertos escenarios de búsqueda textual, pero no deben presentarse como equivalentes a todas las funciones de motores especializados o de MongoDB Search.

Evaluación final

Una persona pide «todas las descripciones que contienen los caracteres `bic` en cualquier posición». Otra pide «documentos relacionados con la palabra bicicleta». ¿Por qué no resolverías automáticamente ambas necesidades con idéntica consulta? Explica qué significa cada petición y cómo comprobarías los resultados.

Laboratorio 17 — Índices y agregaciones: cuando la consulta se convierte en una secuencia de operaciones

Por qué no todas las consultas se escriben con find

Hasta ahora hemos estudiado consultas con `find`, `sort` y `limit`. MongoDB también permite construir **pipelines de agregación**, que son secuencias de etapas por las que pasan los documentos.

Un pipeline puede filtrar, ordenar, limitar, proyectar campos o realizar cálculos. Aprender índices implica saber reconocer cuándo una etapa inicial del pipeline puede beneficiarse de una estructura de acceso.

Vamos a buscar las últimas diez ventas pagadas, pero esta vez expresaremos la operación mediante etapas de agregación para entender que **el índice se relaciona con los filtros y el orden requerido, no con el nombre de la función que escribe el alumno**.

Paso 1. Preparar el servidor

Desde la terminal del ordenador:

```
docker compose up -d --wait mongo-indices
docker compose exec mongo-indices mongosh --quiet
```

Dentro de `mongosh` establece la referencia:

```
const c = db.getSiblingDB("curso_indices").ventas;
```

Comprobar que la colección contiene datos evita investigar planes sobre una base vacía:

```
c.countDocuments({})
```

Paso 2. Formular primero la consulta que ya conoces

```
c.find({estado:"pagada"})
 .sort({fecha:-1})
 .limit(10)
 .toArray()
```

Esta consulta filtra por estado, coloca las fechas más recientes primero y devuelve hasta diez resultados. Copia mentalmente su significado, no todos los documentos de salida.

Paso 3. Expresar lo mismo como pipeline

```
c.aggregate([
  {$match:{estado:"pagada"}},
  {$sort:{fecha:-1}},
  {$limit:10}
]).toArray()
```

Cada etapa recibe documentos de la etapa anterior:

- `$match` descarta las ventas que no están pagadas.
- `$sort` determina el orden por fecha.
- `$limit` limita el número final de documentos.

El resultado expresa la misma necesidad general, pero recuerda que **las fechas iguales pueden producir empates**. Dos planes sin criterio secundario de orden total pueden seleccionar diferentes documentos empatados en el límite.

Paso 4. Consultar el plan de agregación

```
c.explain("executionStats").aggregate([
  {$match:{estado:"pagada"}},
  {$sort:{fecha:-1}},
  {$limit:10}
])
```

La salida puede contener varias estructuras y etapas. No esperes que el JSON sea idéntico al de `find().explain()`. Busca información sobre el acceso inicial a la colección, los índices elegidos y la posible ordenación.

Paso 5. Reconocer por qué importa colocar el filtro pronto

Imagina un pipeline que primero prepara miles de resultados y después decide que solo necesita un estado. Puede trabajar más de lo necesario. MongoDB posee optimizaciones del pipeline que pueden reorganizar operaciones cuando la semántica lo permite, pero **no debemos confiar ciegamente en que cualquier secuencia mal planteada será óptima**.

Explica por qué filtrar por `estado` antes de generar una gran salida suele ser una buena idea conceptual. Después revisa el plan real para ver cómo resolvió MongoDB la consulta.

Paso 6. Añadir una proyección

```
c.aggregate([
  {$match:{estado:"pagada"}},
  {$sort:{fecha:-1}},
  {$limit:10},
  {$project:{_id:0,numero:1,fecha:1,importe:1}}
]).toArray()
```

`$project` escoge los campos del resultado. Hemos decidido no mostrar detalles innecesarios. Eso **no significa automáticamente** que la consulta sea cubierta: necesitaríamos comprobar si el índice contiene todos los datos que exige la ejecución.

Paso 7. Relacionarlo con el índice compuesto

Si existe el índice `{estado:1,fecha:-1,importe:1}` creado en la primera práctica, MongoDB dispone de un acceso potencialmente útil para filtrar por estado y recorrer fechas ordenadas.

No crees un índice adicional solo porque cambiaste `find` por `aggregate`. La pregunta de negocio es esencialmente la misma.

Actividad

Explica la diferencia entre estos tres conceptos:

1. `$match` decide qué documentos cumplen una condición.
2. `$sort` establece un orden.
3. `$project` decide qué campos aparecen en el resultado.

Indica cuál sería un índice candidato para una pantalla que filtra por estado y ordena por fecha, y qué comprobarías mediante `explain` antes de afirmar que ayuda.

Laboratorio 18 — Un error lógico dentro de arrays puede producir resultados falsos

La situación

Una venta puede contener varios productos. La empresa quiere buscar ventas que incluyan un producto de determinado SKU con una cantidad mínima.

Ejemplo:

```
{
  productos:[
    {sku:"SKU10",cantidad:1},
    {sku:"SKU20",cantidad:3}
  ]
}
```

Una consulta que verifica `sku:"SKU10"` y `cantidad:3` por separado sobre elementos del array puede encontrar el documento, aunque **el producto SKU10 tenga solo cantidad uno**. Esto es un error de formulación de la pregunta, no de indexación.

Paso 1. Crear una colección de prueba

En `mongosh`:

```
const d = db.getSiblingDB("curso_indices");
d.productos_pedidos_clase.drop();
d.productos_pedidos_clase.insertMany([
  {_id:1,productos:[{sku:"SKU10",cantidad:1},{sku:"SKU20",cantidad:3}]},
  {_id:2,productos:[{sku:"SKU10",cantidad:3}]},
  {_id:3,productos:[{sku:"SKU30",cantidad:3}]}
]);
```

La colección es pequeña para que podamos predecir manualmente el resultado. La venta uno no debe satisfacer la exigencia de **SKU10 con cantidad tres**, pero la venta dos sí.

Paso 2. Ensayar una consulta ambigua

```
d.productos_pedidos_clase.find({
  "productos.sku":"SKU10",
  "productos.cantidad":3
}).toArray()
```

¿Qué documentos aparecen? Observa que puede encontrarse tanto la venta uno como la dos, porque los predicados se satisfacen en elementos que no necesariamente son el mismo.

Paso 3. Exigir coincidencia dentro del mismo elemento

```
d.productos_pedidos_clase.find({
  productos:{$elemMatch:{sku:"SKU10",cantidad:3}}
}).toArray()
```

Resultado esperado: solo el documento dos. `$elemMatch` obliga a aplicar las condiciones dentro de un mismo elemento del array.

Paso 4. Estudiar un índice que involucre campos del array

```
d.productos_pedidos_clase.createIndex(
  {"productos.sku":1,"productos.cantidad":1},
  {name:"ix_productos_sku_cantidad"}
)
```

En este modelo ambos campos están dentro del **mismo array de subdocumentos**, no en dos arrays paralelos independientes. MongoDB puede construir un índice compuesto multikey apropiado sujeto a las restricciones del modelo.

Paso 5. Leer el plan correcto

```
d.productos_pedidos_clase.find({
  productos:{$elemMatch:{sku:"SKU10",cantidad:3}}
}).explain("executionStats")
```

Busca si utiliza el índice, cuántos documentos encuentra y cuántas entradas examina. No te obsesiones por medir milisegundos sobre tres documentos: la lección importante es **formular correctamente las condiciones y entender las restricciones de un array**.

Paso 6. Entender la limitación

MongoDB no permite indiscriminadamente índices compuestos sobre varios campos que contienen **arrays independientes** en un mismo documento. La restricción evita combinaciones de claves difíciles de manejar y debe tenerse en cuenta en el diseño de esquemas.

Si tu aplicación necesita indexar muchas relaciones complejas, quizá debas reconsiderar la estructura de documentos, no añadir índices cada vez más complicados sin conocer sus reglas.

Evaluación

1. ¿Por qué la primera consulta puede encontrar el documento uno?
2. ¿Qué condición adicional expresa `$elemMatch`?
3. ¿Por qué indexar una consulta incorrecta no arregla su significado?
4. ¿Por qué un multikey puede examinar más claves que documentos?
5. ¿Qué comprobarías antes de crear un índice compuesto que involucre varios arrays?